THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Transport Layer – Services & UDP

Dr Lachlan Andrew

© University of Melbourne 2025

Recap – Application layer protocols

- SMTP, IMAP, POP
- DNS
- HTTP
  - URLs
  - Requests and responses
  - Web servers

Transport layer

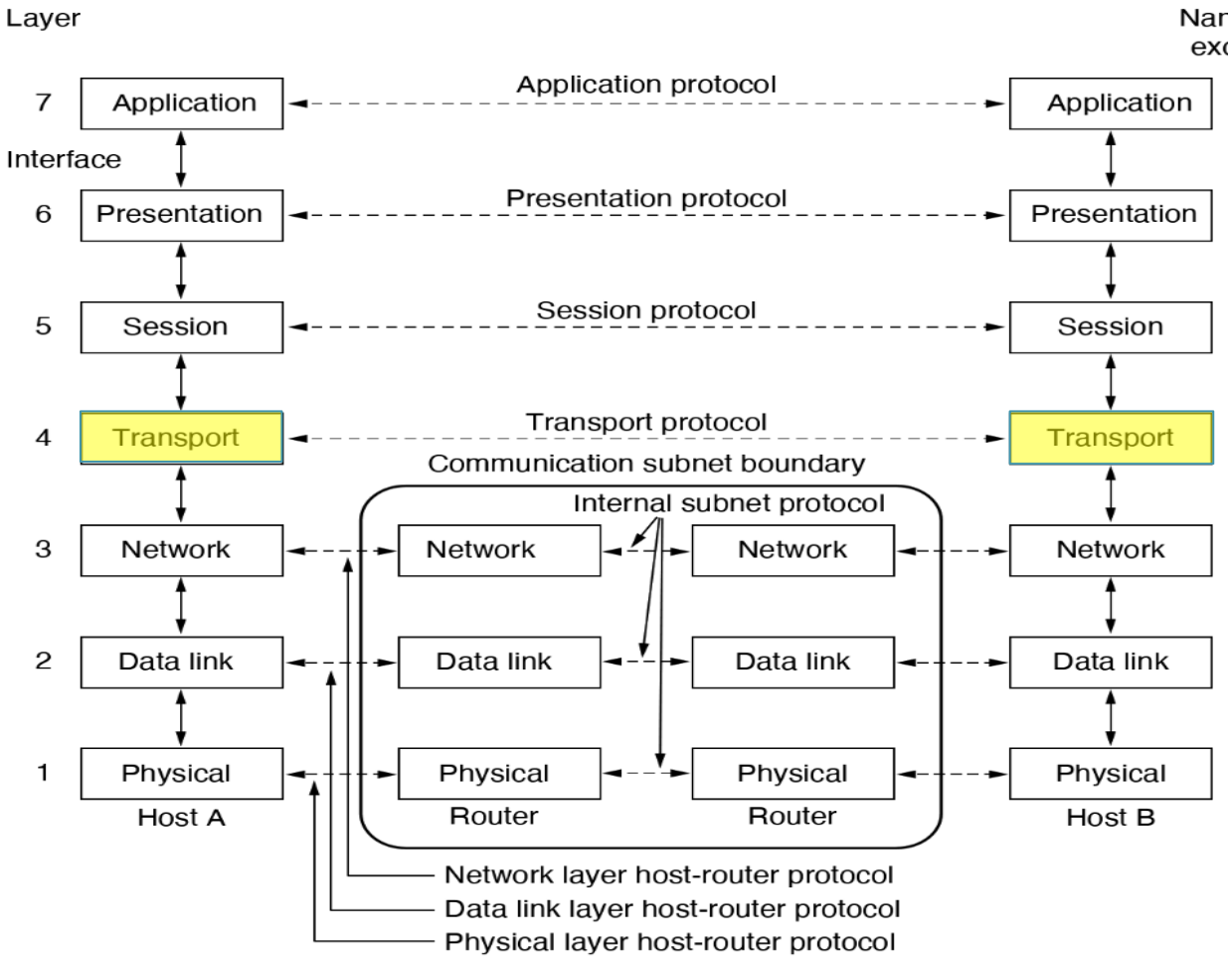
```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

Position in the stack: Network layer



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

Summary

- Transport layer overview
- Services provided
- Simple example: UDP

(This lecture follows naturally from the socket lecture, but we needed to cover other material for the project first.)

Transport layer

Role: provide services needed by applications, using services available by the network layer.

Application needs:

- Data from one application is not mixed with that for another
- Data doesn't arrive faster than we can handle

? Data is streams of bytes

? Data arrives reliably (or we know when a packet has been lost)

? Data arrives in order

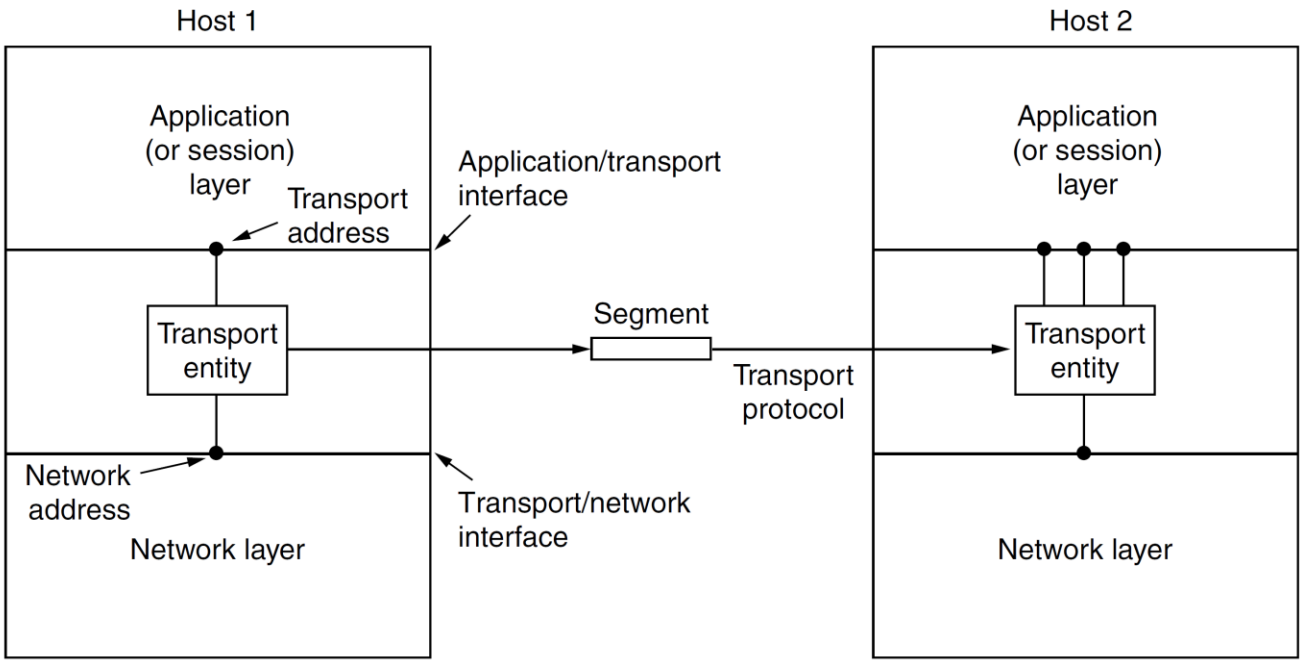
Network provides:

- Get packets from host to host…
  - …most of the time
  - …sometimes multiple copies

Transport Layer

- The transport layer services provide interfaces between the application layer and the network/internet layer.
- Services provide a “logical” communication channel between processes running on different hosts:
  - Connection-oriented
    - = Connection establishment, data transfer, connection release (TCP)
    - Like phone call
  - Connectionless: data transfer (UDP)
    - Like text messages

Transport entity illustrated



TN 6th 6-1

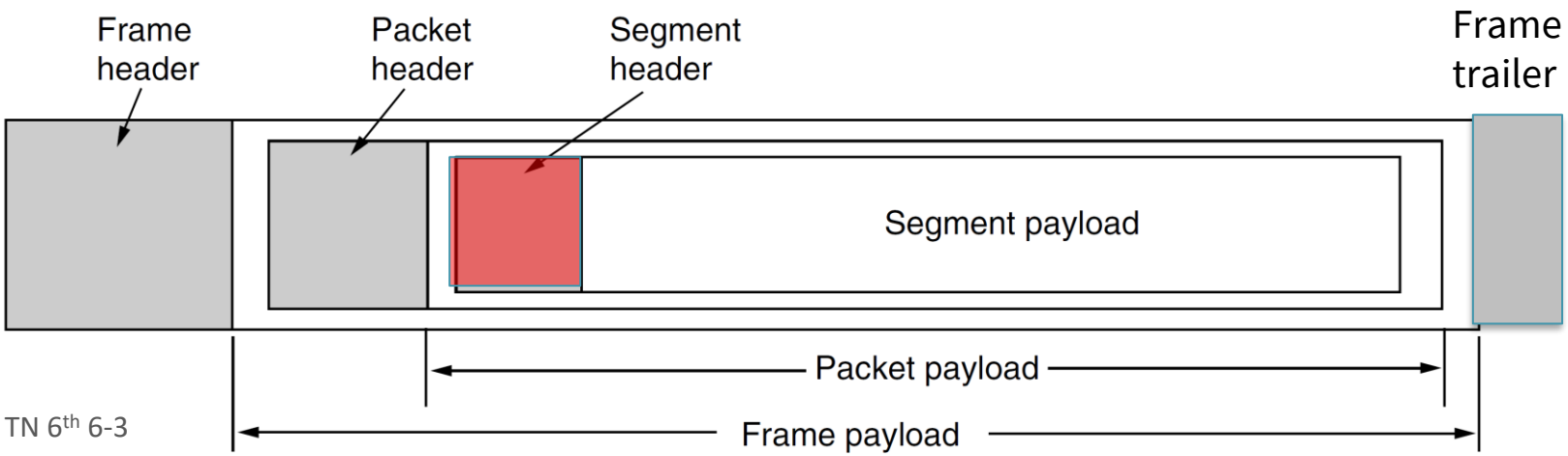
Transport services provide multiplexing, and can provide a reliable service on top of an unreliable network.

Transport layer services

- Terminology (not universal, “packets” also used for all three) :
  - Segments – sent at the transport layer
  - Packets – sent at the internet/network layer
  - Frames – sent at the link/data link layer
- An *unreliable* connectionless service
  - Provides *multiplexing* between different processes
- A *reliable* connection orientated service
  - Provides a notional “perfect” connection between two nodes
    - Doesn’t provide privacy, isochrony (preserving delay between packets)
  - Hides acknowledgements, congestion control, lost packets
  - This service is provided to the higher layers

Transport layer encapsulation

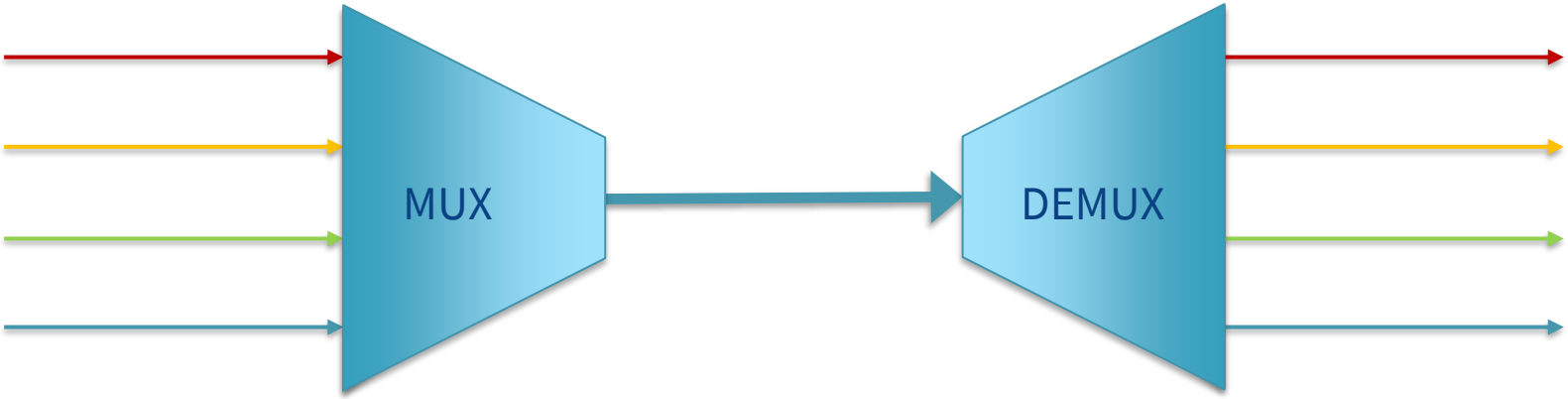
- Abstract representation of messages sent to and from transport entities
- Encapsulation of segments (transport layer units) in packets (network/internet layer units) in frames (data/link layer units)



TN 6th 6-3

Multiplexing /Demultiplexing

- Shortened to MUXING and DEMUXING
  - Multiplexing – combining multiple distinct streams into a single shared stream
  - Demultiplexing – splitting distinct streams out from a single shared stream



Multiplexing: Transport Layer Addressing

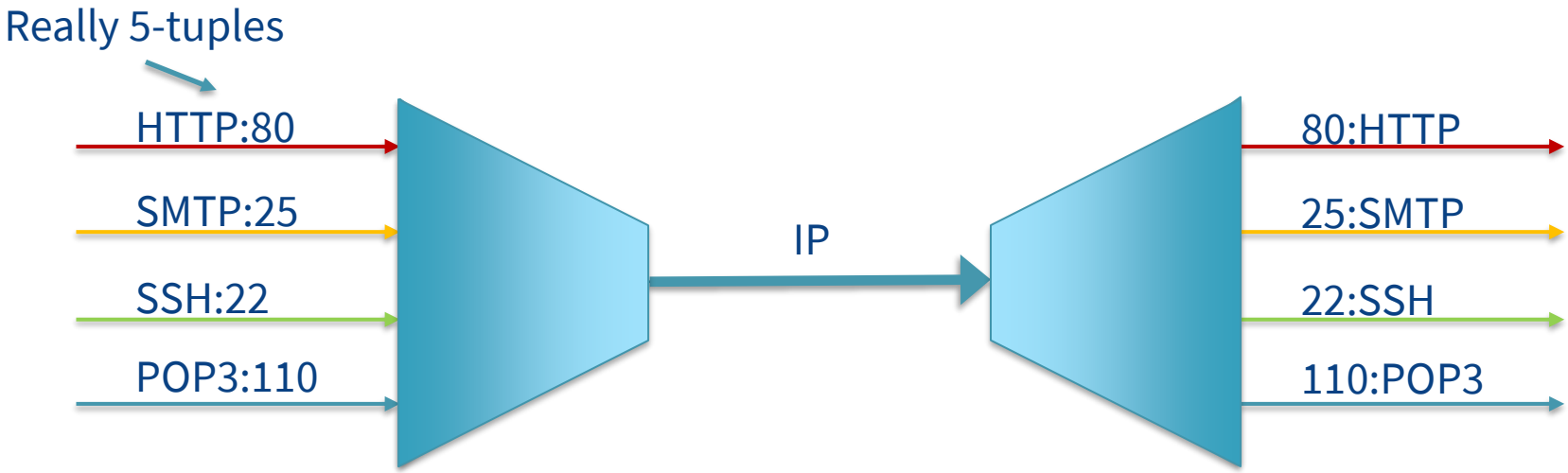
- Specification of the remote process to “connect to” is required at both the application and transport layers.
- Addressing in the transport layer is typically done using *port numbers* (e.g. port 80).
  - Unix /etc/xinetd intercepts incoming connection request packets and spawns a server to handle it, based on the port number.
    - No need to have server always running and accept()ing for rare services.
- Full address is a 5-tuple
  - (protocol, local IP address, local port, remote IP address, remote port)
  - TCP listen sockets and most UDP sockets only have 3-tuple: local IP/port

Port allocations

- Port numbers can range from 0-65535 (16 bits)
- Allocated by Internet Assigned Numbers Authority (IANA)
  - (http://www.iana.org/assignments/port-numbers)
- Ports are classified into 3 segments:
  - Well Known Ports (0-1023)
    - 21 FTP
    - 22 SSH
    - 23 Telnet
    - 25 SMTP
    - 80 HTTP
    - 110 POP3
    - 179 BGP
    - 443 HTTPS
  - Registered Ports (1024-49151)
    - Also called “user ports” but still registered with IANA or similar body
  - Dynamic Ports (49152-65535)

Don’t memorize these. Remember the concept of “well known ports”

MUX/DEMUX


Really 5-tuples
HTTP:80
SMTP:25
SSH:22
POP3:110
IP
80:HTTP
25:SMTP
22:SSH
110:POP3

UDP – User Datagram Protocol

- The User Datagram Protocol allows applications to transmit datagrams over IP without a connection.
  - UDP transmits in segments consisting of a header followed by the payload
- UDP headers contain *source* and *destination* ports
  - UDP socket is a 3-tuple, but each packet has a 5-tuple
- Payload is handed to the process which is attached to the particular port at the destination (using BIND primitive or similar)

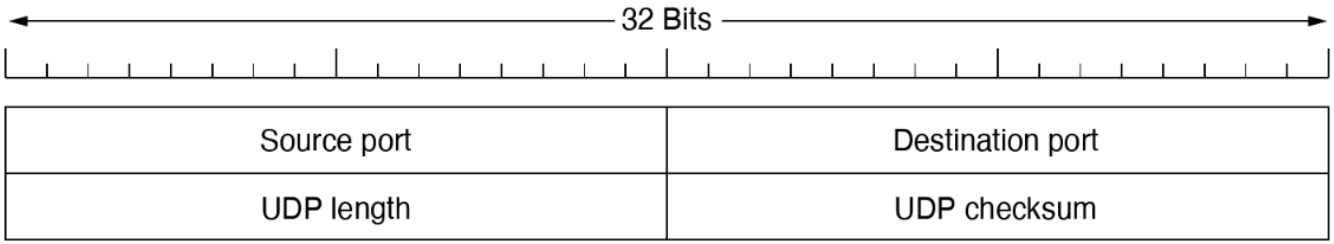
Conversations over UDP

- UDP is connectionless
  - Each message is carried independently
  - (Nothing like “my phone battery died; I’ll switch to the landline”)
- It can still *carry* conversations, like text messages can

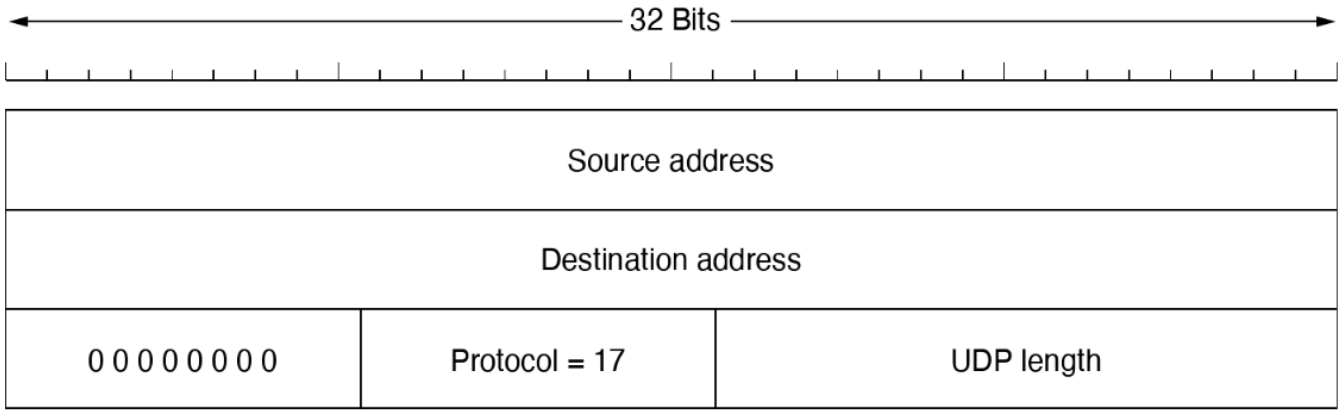
- First message to a well-known port, from a dynamic port
- Server replies from well-known port to same dynamic port
- Client replies using same ports
  - Server can guess it is part of the same conversation, but the *payload* must say where within the conversation it is (like 1/3…3/3 tweets)

UDP header

TN 6th 6-27, 6-28



- (top) UDP header



- (bottom) The IPv4 pseudoheader included in the UDP checksum.

UDP – User Datagram Protocol

- Simple and efficient
- Suitable for some client – server settings
  - Clients sends a short request to the server, expects a short response
  - If that does not occur (request or response is lost) client timeouts and resends
  - Simple to code, and fewer messages, one in each direction
  - DNS is a good example
- Also suitable for real-time services (e.g., VoIP)
  - If a packet is lost, we don't want to wait for it to be resent
  - Loss concealment: fill in the time with our "best guess" sound

UDP pros and cons

Extra service w.r.t IP: UDP provides multiplexing (port numbers)

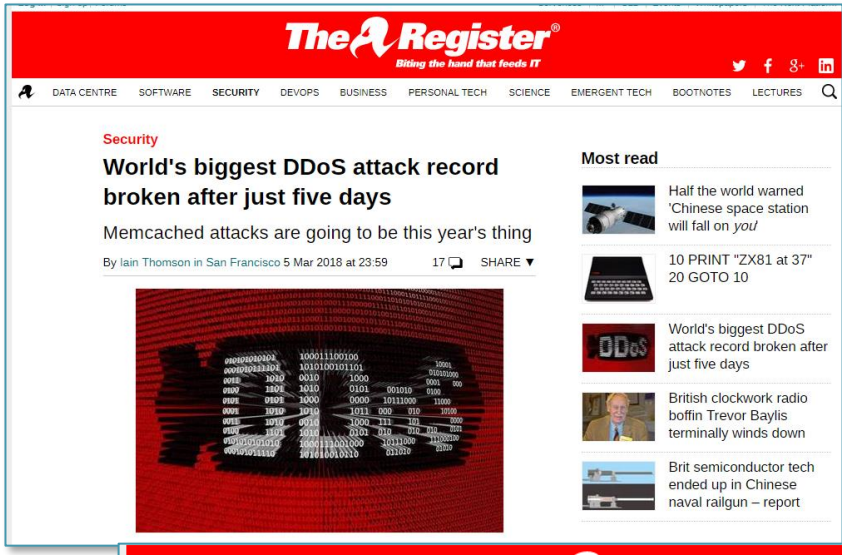
- Strengths and weaknesses of UDP:
  - Strengths: multiplexing/de-multiplexing;
    no delay waiting to recover lost packets
  - Weaknesses: No flow control, error control
    or retransmission of bad segments
  - Conclusion: where applications require a precise level of control over packet flow/error/timing, UDP is a good choice

Class exercise: Multiplexing

- Form groups of six (or more), along a row
- Designate the centre two people as “operating systems”
- Each of the other two, create a UDP socket (allocate port)
- Have a conversation using folded slips of paper
  - One about climate change, one about the post-truth world
- Write the destination port and your own port on the outside
  - How do you know what the destination port should be?
- Repeat without ports: the “OS” randomly hands incoming messages to the two others

And finally…

- Memcached Reflected DDoS Attacks
  - Distributed memory object caching – speeds up dynamic websites by caching database queries
  - Should never been configured externally facing
- Small UDP request made to memcached server with fake source IP
- Memcached responds with up to 50,000 times the data
  - 203 byte request results in 100MB response [https://blogs.akamai.com/2018/03/memcached-fueled-13-tbps-attacks.html]



The Register®
Biting the hand that feeds IT

DATA CENTRE | SOFTWARE | SECURITY | DEVOPS | BUSINESS | PERSONAL TECH | SCIENCE | EMERGENT TECH | BOOTNOTES | LECTURES

Security

World's biggest DDoS attack record broken after just five days

Memcached attacks are going to be this year's thing

By Iain Thomson in San Francisco 5 Mar 2018 at 23:59 17 SHARE ▼

Most read

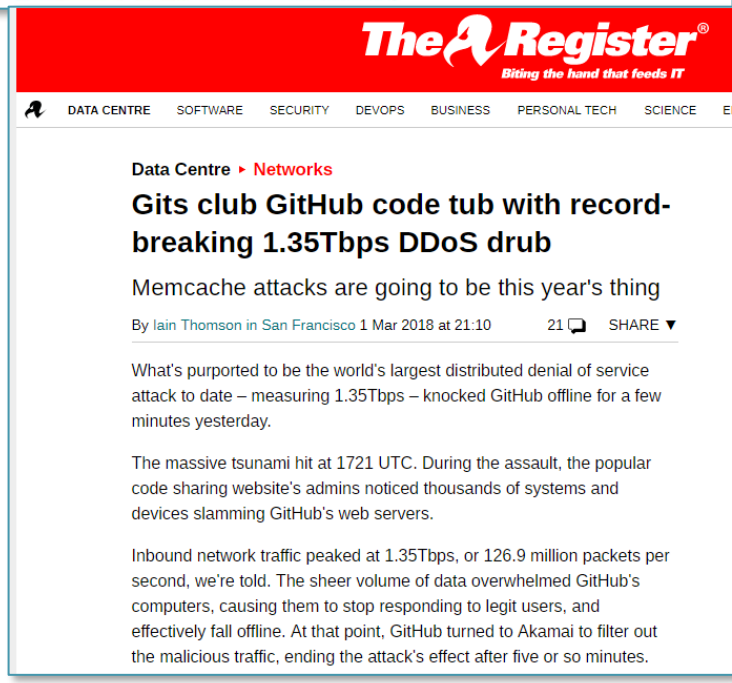
Half the world warned 'Chinese space station will fall on *you*'

10 PRINT "ZX81 at 37" 20 GOTO 10

World's biggest DDoS attack record broken after just five days

British clockwork radio boffin Trevor Baylis terminally winds down

Brit semiconductor tech ended up in Chinese naval railgun – report



The Register®
Biting the hand that feeds IT

DATA CENTRE | SOFTWARE | SECURITY | DEVOPS | BUSINESS | PERSONAL TECH | SCIENCE

Data Centre ▸ Networks

Gits club GitHub code tub with record-breaking 1.35Tbps DDoS drub

Memcache attacks are going to be this year's thing

By Iain Thomson in San Francisco 1 Mar 2018 at 21:10 21 SHARE ▼

What's purported to be the world's largest distributed denial of service attack to date – measuring 1.35Tbps – knocked GitHub offline for a few minutes yesterday.

The massive tsunami hit at 1721 UTC. During the assault, the popular code sharing website's admins noticed thousands of systems and devices slamming GitHub's web servers.

Inbound network traffic peaked at 1.35Tbps, or 126.9 million packets per second, we're told. The sheer volume of data overwhelmed GitHub's computers, causing them to stop responding to legit users, and effectively fall offline. At that point, GitHub turned to Akamai to filter out the malicious traffic, ending the attack's effect after five or so minutes.

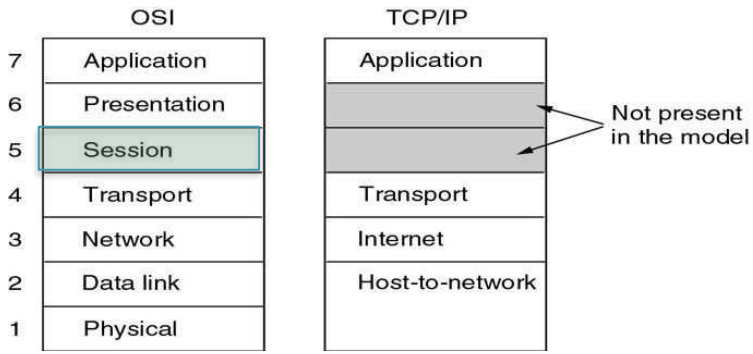
Acknowledgement

- The slides were adapted by Lachlan Andrew from slides prepared by Junhao Gan and Chris Culnane based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed in lecture 1.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879

- Textbook Reference: 3.2,3.3, 9.4.1 (9.3 for interest)

The following slides are *not assessable*, but may be of interest if you are interested in pursuing networking

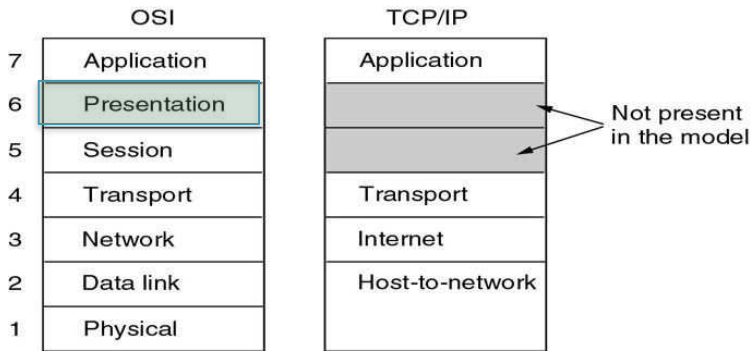
Session Layer



- OSI Layer 5 to provide
  - Authentication
  - Authorization
  - Session restoration
    - continue a failed download
    - log back in to same point in an online purchase
- QUIC does all of these!
- Other examples:
  - Remote procedure call (RPC)
  - *Point-to-point tunneling protocol* (PPTP)
  - *Password (/Extensible) Authentication Protocol* (PAP/EAP)
- *These* often used between protocols called* layer 2 and layer 3

*Layers are funny. Ethernet is always called "layer 2", but has many properties of layer 3, and even some of layer 4.

Presentation Layer

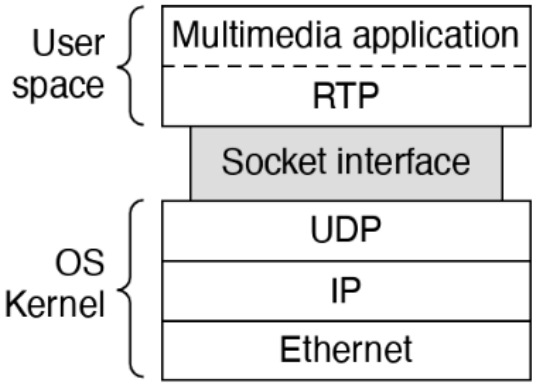


- OSI layer 6 to provide:
  - Encryption
  - Compression
  - Data conversion (e.g., mapping CR/LF to LF, .doc to .docx)
  - Mapping between character sets (ASCII/EBCDIC, now UTF-8/BIG5/…)
- These services haven't vanished: done by applications
- Why does IETF consider them "Application Layer"?
  - The protocol to negotiate encryption etc. is quite simple and separate from the algorithms
  - There aren't simple "common services" needed by all applications
  - The application is not in the kernel, and so much more flexible
  - "Layering violations"
- Closest thing to presentation layer: Real time protocol (RTP)

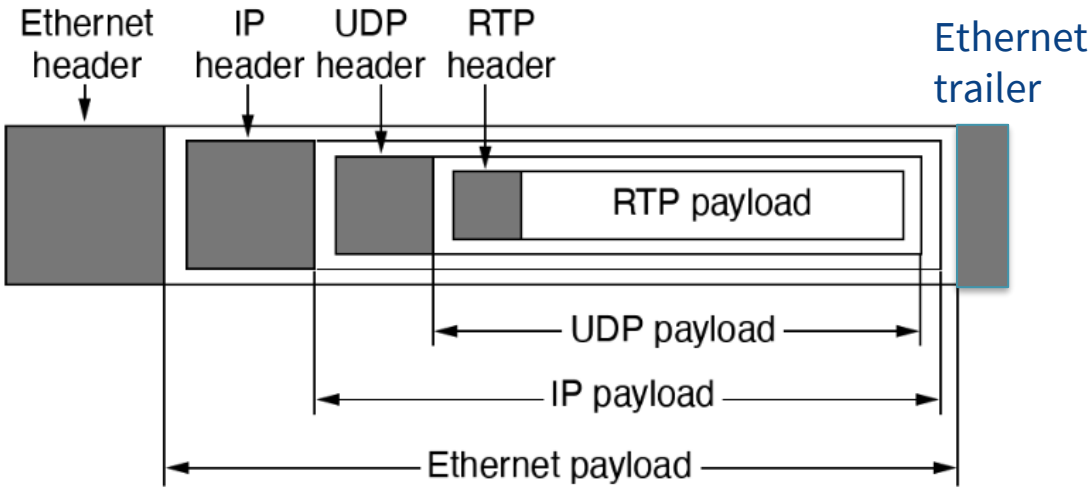
RTP – Streaming and VOIP

- Real-Time Transport Protocol (RTP)
- Which layer is RTP at? IETF person says:
  - Runs in user space, uses UDP from the transport layer -> Application layer
  - Generic protocol that provides services to applications -> Transport layer
  - (OSI person says: Neither – Presentation layer!)
- RTP multiplexes several streams into a single stream of UDP segments

RTP in context



(a)



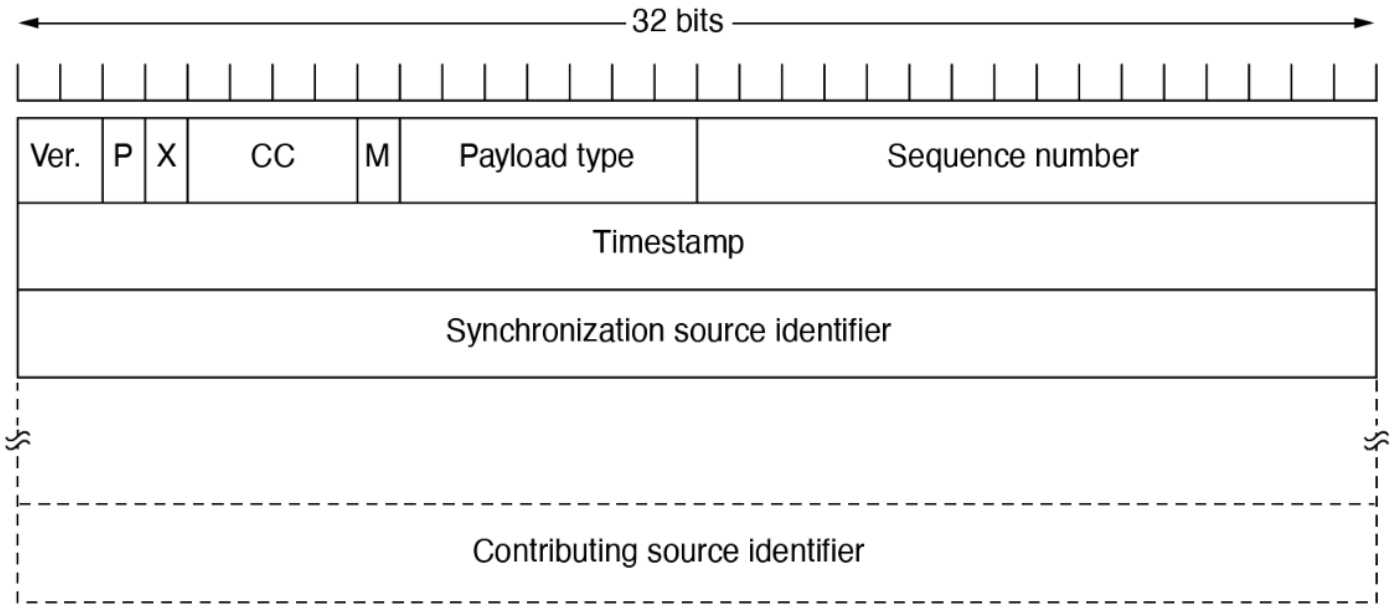
(b)

TN 6th 6-30

Packet nesting

The position of real-time protocol in the protocol stack

RTP Header



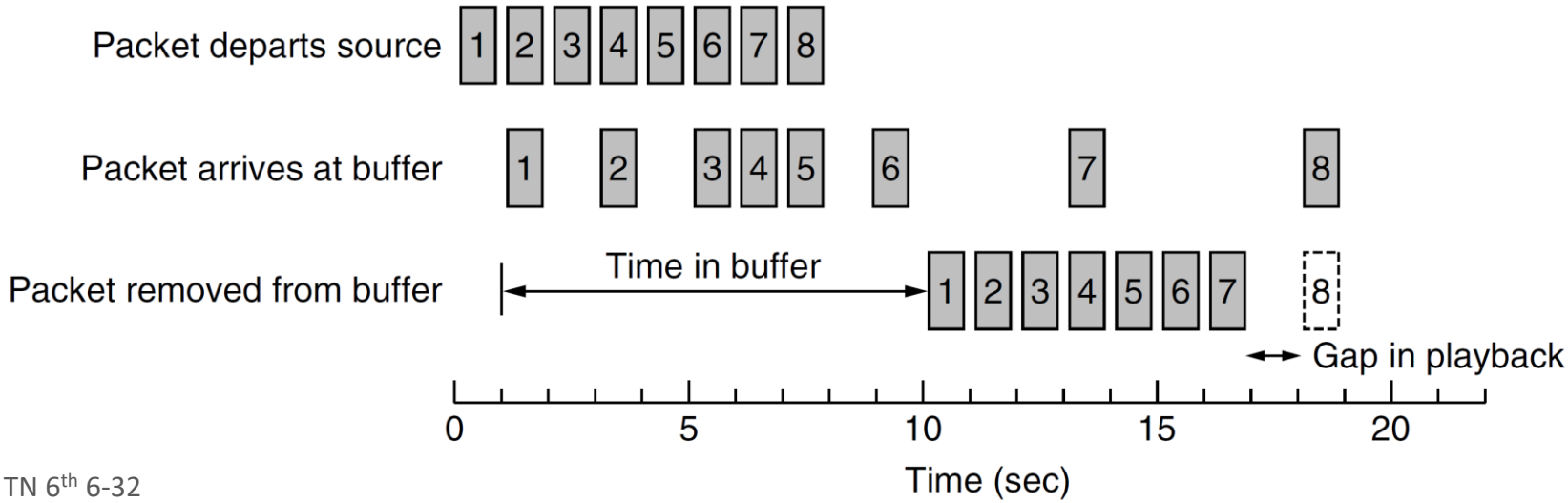
TN 6th 6-31

- Payload type – encoding used (MP3, etc.) – can vary each time
- Sequence Number – counter incremented on each packet
- Timestamp – Source controlled relative to start of the stream

Real-time Transport Control Protocol (RTCP)

- Control protocol for RTP
  - Handles feedback, synchronization, and UI
- Feedback to source
  - Delay, jitter, bandwidth, congestion
  - Used by encoder to adaptively encode to suit network conditions
  - In multicast settings, feedback is limited to small percentage of media bandwidth
- Synchronization
  - Where different streams use different clocks/have different drift
- UI
  - naming sources to show who is on a conference call
- (Another network model:
  "Control plane" is a stack parallel to the "data plane" stack.)

RTP Playback



TN 6th 6-32

- Jitter – variation in delay of packets
  - Buffer at receiver to counter it
- Packet 8 too late, can wait or skip, depending on application
- Size of buffer is also application specific (VOIP = small buffer)